BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .

3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is basically the whole map of the problem — every possible situation you could be in, and the moves that get you from one to another. In our grid, that's every cell you could stand on and the Up/Down/Left/Right moves connecting them. It just exists; it doesn't change no matter what the agent does, and each cell only shows up once. The search tree is what the algorithm actually builds as it searches. It starts at your starting cell and branches out as you explore. The important thing is that each node in the tree is a

path to a cell, not the cell itself. So the same cell can show up many times in the search tree because there are many different routes to reach it. Simple version: the state space is the map, the search tree is the record of the paths you tried while exploring that map.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set just keeps track of which cells we've already visited. We need it because the grid has loops you can go Right then Left and end up right back where you started. Without something to remember where you've been, the search keeps re-visiting the same cells again and again. If DFS didn't have the reached set, it would get stuck infinity loop it'd keep bouncing between the same cells forever and never finish or find the food. The reached set is what stops that from happening, and it's the thing that turns a plain "tree search" into a proper "graph search."

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS explores in layers — first all the cells 1 step away, then all the cells 2 steps away, and so on. Because of that, the moment it reaches the food, it's guaranteed to have used the fewest possible moves. And since every move in our grid costs the same (1 step), fewest moves = shortest path. That's why BFS is always optimal here. DFS works differently — it picks one direction and keeps going as deep as possible before backing up. It just takes the first path it happens to find to the food, not the shortest one. That's why its route looks messy and winding on screen, doubling back on itself. DFS was never trying to find the shortest path; it only cares about going deep.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

The big difference is how much memory each one uses BFS has to remember every cell in the current layer it's exploring, plus everywhere it's already been. On a giant grid that's up to a million cells sitting in memory at once, so BFS (and UCS, which is similar) can run out of memory and crash. That huge growing frontier is the problem. DFS only has to remember the single path it's currently on from the start down to wherever it is now. That's just a few thousand cells at most, way less than a million. So DFS uses far less memory. The catch is that DFS pays for this by not guaranteeing the shortest path Both take similar time in the worst case — the real difference here is memory, and that's where DFS wins.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.